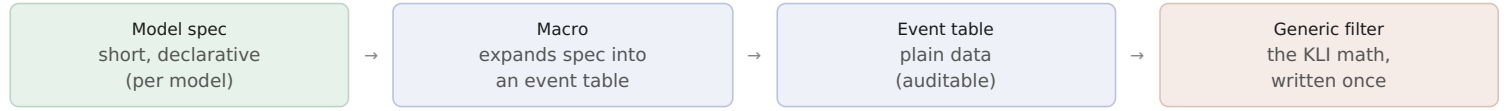


One filter, many models — technical companion

Implementation sketch for a model DSL over a single generic KLI filter · [PhyloPOMP.jl](#)

Read this first. The code below is an **illustrative design sketch**, grounded on the [kingaa/PhyloPOMP.jl](#) devel branch as read on 2026-07-07. It has **not** been executed (the sandbox can't reach Julia's binaries), so treat the surface syntax as indicative, not compiler-checked. Crucially: the three weight-bearing functions in §5 are **slots for the King-Lin-Ionides (KLI) reduction**. Their bodies are where the boost / decay / compatibility algebra goes, and that algebra must be verified against the paper — this document locates the work, it does not certify it.

1 · The architecture in one row



The design principle from our discussion: **the macro carries no likelihood math**. It only turns a readable spec into an event table. The KLI reduction — regular part, singular part, boost, decay, driver, compatibility, the combinatorial ℓ -factors — lives once in the generic filter and is parameterised by that table. Adding a model is a spec change; the audited math is untouched.

2 · The data model

An event is plain data. A model is a list of events plus its two vocabularies (all compartments; the deme subset that carries lineages — matching `Coloring` being built over the subset, e.g. `{Expos, Infec}`, not all of `{S, E, I, R}`).

```
@enum EventType BIRTH MIGRATION DEATH SAMPLE NEUTRAL # the five pure event types, KLI §3.2

struct Event
    name      :: Symbol
    Δ          :: Vector{Pair{Symbol, Int}} # population stoichiometry, e.g. [:S=>-1, :E=>+1]
    hazard     :: Function                  # (x, θ) -> Float64 ; x, θ are NamedTuples
    r          :: Vector{Int}              # production r^u — a per-mark CONSTANT (§3.3.1, Table 2)
    type       :: EventType                # birth/migration/death/sample/neutral (§3.2)
    from       :: Int                      # source deme (birth parent / migration source); 0 = none
    into       :: Vector{Int}              # child / target deme(s)
    regular    :: Bool                     # regular (between events) vs singular (at ev(Z))
    observed   :: Bool                     # produces a genealogy feature vs background
end

struct MGPMModel
    name      :: Symbol
    compartments :: Vector{Symbol} # full population state, e.g. S,E,I,R
    demes      :: Vector{Symbol} # lineage-carrying subset -> this is the Coloring set
    events     :: Vector{Event}
end
```

Why `r` is stored but `s` and `ℓ` are not. Production `ru` is a per-mark constant (§3.3.1, Table 2), so it belongs on the event. Saturation `s` and lineage count `ℓ` are *not* constants — both are functions of the pruned genealogy at `t` (`s = s(Yt)`, §3.4.2), so the filter computes them at each event and never stores them. The specific coloring operator (chop χ , swap σ , fork κ ; §5.1) is likewise *derived* at runtime from `type` and the local genealogy via the compatibility indicator `Qu` (Eq. 38): a single event mark is generally compatible with several coloring outcomes, so it cannot be one stored operator.

3 · The event table with no macro (the audit target)

Before writing any macro, hand-write what it must expand to — this explicit table is the object you diff against KLI's event construction and against `event_rates!`.

```
SEIR = MGPMModel(:SEIR, [:S,:E,:I,:R], [:E,:I], [ # demes ordered (E,I): deme 1=E, 2=I
    Event(:infection, [:S=>-1,:E=>+1], (x,θ)->θ.β*x.S*x.I/θ.N,
        [1,1], BIRTH, 2,[1], true, false), # r^u=(1,1) Table 2; I parent sires E child
    Event(:progression, [:E=>-1,:I=>+1], (x,θ)->θ.σ*x.E,
        [0,1], MIGRATION, 1,[2], true, false), # r^u=(0,1) Table 2; E → I
    Event(:recovery, [:I=>-1,:R=>+1], (x,θ)->θ.γ*x.I,
        [0,0], DEATH, 2,Int[], true, false), # r^u=(0,0); no coloring op — decay only
    Event(:waning, [:R=>-1,:S=>+1], (x,θ)->θ.ω*x.R,
        [0,0], NEUTRAL, 0,Int[], true, false),
    Event(:sampling, Pair{Symbol,Int}[], (x,θ)->θ.ψ*x.I,
        [0,1], SAMPLE, 2,Int[], false, true), # r^u=(0,1) Table 2; chop χ
])
```

If you can check that table row-by-row against the paper and against `seir_naive.jl`, the DSL has done its whole job. Everything else is convenience.

4 · The macro

Surface syntax uses space-separated `key = value` arguments, which the parser hands over as plain assignment expressions — no fragile operator parsing:

```
@mgp SEIR begin
    compartments = (S, E, I, R)
    demes        = (E, I)
    params       = (β, σ, γ, ω, ψ, χ, N)

    @event infection rate=β*S*I/N pop=(S=-1, E=+1) move=fork(I => E,I) kind=regular
```

```

@event progression rate=σ*E      pop=(E=-1, I=+1) move=swap(E => I)      kind=regular
@event recovery   rate=γ*I      pop=(I=-1, R=+1) move=chop(I)        kind=regular
@event waning     rate=ω*R      pop=(R=-1, S=+1) move=none          kind=regular
@event sampling   rate=ψ*I      pop=()      move=sample(I)          kind=singular
end

```

The macro has exactly three jobs.

(a) Rewrite bare model symbols into field accesses

This is the one genuinely useful bit of metaprogramming and the reason it's a macro, not a function: $\beta S^*I/N$ must become a closure that reads compartments from the state and parameters from θ . With `MacroTools`:

```

using MacroTools: postwalk

rewrite(ex, comps, pars) = postwalk(ex) do n
  n isa Symbol || return n
  n in comps && return :(x.$n)      # compartment -> x.field
  n in pars && return :(θ.$n)      # parameter   -> θ.field
  n              # operators / literals: leave alone
end

make_hazard(rate_ex, comps, pars) = :( (x, θ) -> $(rewrite(rate_ex, comps, pars)) )

```

(b) Decode the coloring move

`move=fork(I => E,I)` becomes `(FORK, from, into)` by matching the call head against the five operators and mapping deme symbols to their indices in the declared `demes` tuple.

(c) Emit the enums and assemble the model

```

macro mgp(name, block)
  comps = pars = demes = Symbol[]; events = Expr[]
  for stmt in block.args
    stmt isaLineNumberNode && continue
    @capture(stmt, compartments = (cs__)) && (comps = cs; continue)
    @capture(stmt, demes = (ds__)) && (demes = ds; continue)
    @capture(stmt, params = (ps__)) && (pars = ps; continue)
    @capture(stmt, @event(args__)) && push!(events, build_event(args, comps, demes, pars))
  end
  quote
    @demes $(Symbol(name, :Demes)) $(demes...)      # reuse the shipped @demes macro
    const $(esc(name)) = MGPMModel($(QuoteNode(name)), $comps, $demes, [$(events...)])
  end
end

```

Hygiene caveat. The one place to get wrong is escaping: the emitted closures reference the user's β , S etc., so the rewrite must run before hygiene renames them, and the final binding `(esc(name))` is escaped so `SEIR` lands in the caller's scope. Verify this by `@macroexpand @mgp SEIR` ... and confirming it reproduces the §3 table verbatim.

5 · The generic filter (where the KLI math lives, once)

The loop structure is safe to write generically. The three marked calls are the KLI reduction and are the *only* places the boost / decay / compatibility algebra appears — for any model.

```

function regular_step!(cols, ll, t, dt, x, model, θ)
  t0 = t
  while t < t0 + dt
    α = [ev.regular ? ev.hazard(x, θ) : 0.0 for ev in model.events] # driver / population rates
    π = [kli_select(ev, cols, x) for ev in model.events]           # ① KLI-traced
    k, s = rcateg(α .* π)
    step = -log(rand()) / s
    if t + step < t0 + dt
      ll -= kli_decay(α, π, cols, x, model) * step + log(π[k])      # ② KLI-traced
      apply_pop!(x, model.events[k])                                # stoichiometry only
      Δw = apply_move!(cols, model.events[k], x)                   # ③ returns KLI boost
      ll += Δw
      t += step
    else
      ll -= kli_decay(α, π, cols, x, model) * (t0 + dt - t)
      break
    end
  end
  ll
end

```

The **singular** update is the same shape: dispatch on node type (branch-point $\rightarrow$ fork κ , sample $\rightarrow$ chop χ , inline $\rightarrow$ swap σ ; §5.1) crossed with `observed`, and call the *same* weight functions so the observed-event boost and the sum over compatible latent assignments are computed in one audited place. Roots are not an operator: they enter through the filter's initial condition $w(\theta, x, y) = p_\theta(x) \mathbb{1}\{q(x, y) > 0\}$ (Theorem 5).

6 · KLI traceability — what each slot absorbs

Every hand-typed correction scattered across `seir_naive.jl` maps to one generic slot. The right-hand column is now pinned to specific equations in the uploaded KLI paper, verified against the text rather than from memory.

Generic slot	What it computes	Absorbs these hand-written lines	KLI object · equation (verified)
ev.hazard	the population hazard α_u (one half of the driver $\beta_u = \alpha_u \pi_u$)	$\alpha[k] = \beta * S * I / \text{pop}$, $\sigma * E$, $\gamma * (I - \text{ell}I)$, $\omega * R$	hazard α_u (§2.5); r^u , type in Table 2; driver β_u in Thm 5 & Eq. 45 · mechanical
kli_select	the importance kernel π_u — probability of proposing which lineage/branch the event acts on	$\pi[k] = \text{ell}I / I$, $1 - \text{ell}I / I$, $\text{ell}E / E$, ...	kernel π_u , normalization Eq. 44; naïve $\pi = 1/E$, $1/I$, $1/2$ (text, p. 27) · verify choice
kli_decay	the decay λ integrated over each inter-event interval	event_rates! return: $\psi * I + \chi * I + \gamma * \text{ell}I + \dots$; $\text{ll} := \text{decay} * \text{step}$	decay λ Eq. 47 (SEIRS); general form Eq. B2 · to verify
apply_move! (Δw)	the multiplicative boost on a coloring move (chop χ / swap σ / fork κ)	$\text{ll} += \log(\text{ell}I)$; $\text{ll} -= \log(E)$; $\text{ll} -= \log(E * I)$; $\log(1 - \text{ell}I / I) - \log(E)$	boost $B = \Psi_u = \varphi_u / \pi_u$ (Thm 5); φ_u Eq. 9; binomials Eq. 37; Q_u Eq. 38; SEIRS boost Eq. 46 · to verify
singular boost	the boost applied at an observed tree feature (branch, tip)	$\log(\beta * S * I / \text{pop}) \dots - \log(E * I)$; $\log(\psi * I) + \log(1 - \text{ell}I / I)$; $\log(\chi * I)$	singular part Eq. 14 (Thm 5); SEIRS singular Eqs. 39–41; general Eq. B3 · to verify

The naïve / guided / hard tracks differ only in the proposal kernel π (Eqs. 44–46); the guided / relhaz track builds an anticipatory π from a reverse-time sweep over the deme process (text p. 27; Discussion §6) — still the **same forward** SMC filter (Lemma 3 / Algorithm B1), not the adjoint. The adjoint filter (Corollary 6, Eqs. 15–16 / B11–B12) is a separate *backward* reformulation that computes the same L ; the forward scheme here does not use it.

This is exactly where your MERS defects live — now anchored. The missing $+\log(\ell)$ on a regular cross-deme swap is the ℓ_1 -multiplier in the summed-over- m transmission term (SEIRS Eq. 43), which the Chu–Vandermonde identity (Eq. 8) collapses; it lives in kli_select / apply_move!. The missing $+\log(2)$ on a singular symmetric fork is the two-orientation branch-point boost (SEIRS Eq. 41, $t \in B$), equivalently the binomial ratio $\binom{\ell}{\ell/2} = 2/(I(I-1))$ in the one-deme SIRS case (Eq. 37; SIRS Eq. 32); it lives in apply_move!. Transposed birth rates are an ev.hazard / Table 2 error.

Correction from checking the source. A prior note tied the SEIRS filter to “Eqs. 36–39.” In the uploaded paper, **Eq. 36 is the one-deme SIRS regular filter**, not SEIRS. The SEIRS filter is Eqs. 39–47: singular Eqs. 39–41, regular Eqs. 42–43, with driver/boost/decay named explicitly in Eqs. 45/46/47 and kernel normalization in Eq. 44. Worth re-pinning wherever that reference was used.

7 · What this buys, and what stays manual

Buys:

- The boost/decay/compatibility algebra is written and audited **once**. A dropped `log`-term can occur in one function, not in every branch of every model.
- β_{21} vs β_{12} directionality is one labelled `hazard` line per event — a transposition is visible, not buried in C indexing.
- Regular vs singular is a field, so an event can't be silently filtered as a pointwise hazard when it owes a sum over latent assignments.
- `@macroexpand` yields the exact event table — your KLI event-table audit step becomes a diff, not a re-derivation.

Stays manual (unchanged by this design):

- Deriving and verifying the five slot bodies against KLI — once, for all models.
- The proposal spectrum (naïve / guided / hard) is a second axis; it parameterises the same generic filter rather than spawning new bodies, but each kernel still needs its own verification.
- The reverse-time guide (`fsmarkov`) for the guided proposal is real linear-algebra work that this DSL does not remove.

Bottom line. A macro turns a short per-model spec into an auditable event table; one generic filter consumes it. The number of places a KLI term can be wrong drops from “every branch of every model” to “five named functions” — which is exactly the surface your audits keep having to re-cover.